# 2. Tech Stack & Component Architecture

All technology dependencies, version numbers, component modules, and architectural rationale have been verified against the codebase manifests (`package.json`, `node_modules`, and component trees):

---

## 2.1 Frontend Architecture & Component Ecosystem

The frontend client is built on Next.js 16 using the App Router model, structured into modular client/server component hierarchies, custom UI design tokens, context providers, and dynamic route handlers:

- **Framework & Core Engine**:
  - **Next.js (`v16.2.10`)**: App Router architecture utilizing nested layouts (`app/layout.tsx`, `app/admin/layout.tsx`, `app/dashboard/layout.tsx`), route grouping, server-rendered page shells, and client-side page transitions.
  - **React (`v19.2.4`) & React DOM (`v19.2.4`)**: Core component UI engine rendering declarative states, reactive context hooks, and DOM portals for modal overlays.
- **Styling, Design System & Theming**:
  - **TailwindCSS (`v4.0.0`) & `@tailwindcss/postcss` (`v4.0.0`)**: Utility-first CSS framework coupled with custom CSS variables in `app/globals.css`.
  - **Design System**: Built with modern visual tokens including custom HSL color palettes, subtle glassmorphism (`backdrop-blur-md`, semi-transparent borders), glowing accent highlights, and dark-mode aesthetic styling.
  - **Theme Management (`components/layout/ThemeProvider.tsx`)**: Client-side theme provider supporting dynamic dark and light mode switching with

persisted theme state.

- **Animations & Micro-Interactions**:
  - **Framer Motion (`v12.43.0`)**: Powers interactive UI animations including smooth card hover scaling, layout transitions, modal backdrop fade-ins, filter tab sliding indicators, and dynamic feedback toasts.
- **Frontend Component Modules**:
  - **Core UI Primitives (`components/ui/`)**:
    - `Button.tsx`: Customizable button component supporting variants (`primary`, `secondary`, `outline`, `danger`, `ghost`) and loading state spinners.
    - `Input.tsx`: Accessible form field input with floating label animations, error boundary highlights, helper text, and show/hide password toggles.
    - `Modal.tsx`: Accessible portal dialog overlay with backdrop blur, keyboard ESC dismissal, and focus trapping.
    - `Badge.tsx`: Visual tag component displaying entity statuses (`pending`, `approved`, `rejected`, `sold`, `verified student`).
    - `StatCard.tsx`: Metric card for analytics dashboards displaying total counters, icons, and trend indicators.
    - `EmptyState.tsx`: Graphic placeholder component displayed when zero search results, listings, or notifications are found.
    - `Pagination.tsx`: Paginated navigation controls for large data tables and listing grids.
    - `Skeleton.tsx` & `Spinner.tsx`: Loading shimmer blocks and spinning indicators for asynchronous data fetches.
  - **Custom Loading Primitives (`components/loading-ui/`)**:
    - `twin-orbit.tsx` & `fade-arc.tsx`: Custom SVG-based animated loading graphics used during high-level layout mounts and route transitions.

    - **Layout & Navigation (`components/layout/`)**:
        - `Navbar.tsx`: Sticky navigation header featuring search query input triggers, category navigation links, dynamic role-based navigation (Buyer / Seller / Admin links), user avatar dropdown, and mobile navigation drawer.
        - `Footer.tsx`: Universal footer featuring platform mission, category links, legal disclosures, and contact links.
    - **Marketplace Components (`components/marketplace/`)**:
        - `ProductCard.tsx`: Marketplace grid card featuring product preview image carousel, price formatting in ETB, category pill, verified student seller badge, and hover action triggers.
        - `ProductCardSkeleton.tsx`: Multi-card skeleton loader preserving layout geometry while fetching marketplace listings.
        - `FilterBar.tsx`: Dynamic search and filtering control bar supporting full-text search input, category tabs, product type dropdowns, and sorting order selectors.

- **State Management & Authentication Context (`contexts/AuthContext.tsx`)**:
    - Centralized React context managing user authentication state, JWT access token storage, automatic token refreshing, user profile state, session termination, and client-side route guard protection.

- **Forms, Data Utilities & Iconography**:
    - **React Hook Form (`v7.82.0`)**: Form state tracking, client-side input validation, error handling, and performance-optimized input binding.
    - **Date Utilities (`date-fns v4.4.0`)**: Date parsing and relative time formatting (e.g. "2 hours ago", "Member since 2026") across user activity feeds, transactions, and reviews.
    - **Lucide React (`v1.25.0`)**: Clean, consistent SVG icon

library used across navigation bars, buttons, status indicators, and metric dashboards.
    - **TypeScript (`v5.x`)**: Strict static typing across API DTO payload interfaces, context values, Prisma models, and component props.

---

## 2.2 Backend API Architecture & Technologies

The backend API is built as a modular NestJS server (`v11.1.28`), exposing RESTful controllers, decoupled domain services, custom authentication guards, and validation pipelines:

- **Framework & Modular Architecture**:
    - **NestJS (`v11.1.28`)**: Structured around `@nestjs/core`, `@nestjs/common`, `@nestjs/platform-express`, and `@nestjs/config` (`v4.0.4`).
    - **Feature Modules**: Organized into decoupled domain modules: `AuthModule`, `UsersModule`, `ProductsModule`, `CategoriesModule`, `ProductTypesModule`, `AdminModule`, `TransactionsModule`, `UploadsModule`, and global `PrismaModule`.
- **Database Engine & ORM Rationale**:
    - **Prisma Client & CLI (`v7.8.0`)**: Type-safe database ORM managing query execution, relational inclusions (`include: { seller: true, images: true }`), and transactional atomic operations.
    - **PostgreSQL 16 Engine (Supabase Cloud & Docker)**:
        - *Deployment & Target Rationale*: PostgreSQL 16 hosted on **Supabase Cloud** for the deployed production environment (Docker-based local Postgres container used during local development). Supabase Cloud Postgres was selected specifically to support live deployment on Vercel (frontend) and Railway (backend) — a locally-hosted database cannot be reached by external cloud hosting platforms, making a

managed cloud Postgres instance a mandatory requirement of the deployment target architecture, not a deviation from it.

- **Authentication & Security Infrastructure**:
  - **Passport JWT (`@nestjs/passport v11.0.5`, `@nestjs/jwt v11.0.2`, `passport-jwt v4.0.1`, `passport v0.7.0`)**: Implements stateless JWT authentication using Bearer tokens.
  - **Bcrypt (`v6.0.0`)**: Salted password hashing (12 rounds) for registration and password changes.
  - **Guards & Authorization**:
    - `JwtAuthGuard`: Extracts and verifies access tokens.
    - `RolesGuard` & `@Roles('admin')`: Custom metadata decorator enforcing role-based access control on sensitive endpoints.
    - **Active Database Account Guard**: On every request, `JwtStrategy` queries the database (`prisma.user.findUnique`) to verify account status. If `is_banned` is `true`, the strategy immediately throws a `403 ForbiddenException`, instantly revoking access even for previously issued valid JWT tokens.
  - **Rate Limiting (`@nestjs/throttler v6.5.0`)**: Throttles API endpoint abuse.

- **Data Validation & DTO Layer**:
  - **`class-validator` (`v0.15.1`) & `class-transformer` (`v0.5.1`)**: Global NestJS `ValidationPipe` enforces incoming payload schemas across DTOs (`CreateProductDto`, `RegisterDto`, `BanUserDto`, etc.), validating string bounds, numeric constraints, array item types, and optional parameters before controller execution.

- **File & Media Storage Pipeline**:
  - **Cloudinary Node SDK (`v2.10.0`)**: Integrated with NestJS `FileInterceptor` via Express Multer memory storage buffers.

    - Handles image upload endpoints (`POST /uploads/image`) with image mimetype filters (JPEG, PNG, WEBP, Max 5MB) for profile avatars and product listing photos, as well as file upload endpoints (`POST /uploads/file`, Max 10MB) for gated deliverable files.

- **Payment Processing Strategy & Regional Market Rationale**:
    - **Internal Synchronous Mock Checkout**: Handled via an internal, synchronous mock transaction workflow (`is_test: true`, instant `completed` status) by deliberate design.
    - *Market & Regional Rationale*: Credit-card-based payment platforms (such as Stripe) have negligible adoption among the platform's target university student demographic in Ethiopia. Stripe was evaluated and intentionally not integrated for this reason.
    - *Architecture & Future Provider Handoff*: The full purchase-to-delivery pipeline (transaction recording, self-purchase validation, product access state update, deliverable file unlocking, seller sales tracking, buyer reviews) is fully built, tested, and operational against this mock trigger. The backend transaction service is explicitly architected so the trigger point can be swapped for a real payment provider's webhook confirmation event (specifically **Telebirr**, Ethio Telecom's dominant mobile money platform) without requiring any structural changes downstream.